

Université Hassan II de Casablanca
ENSET Mohammedia

Master SDIA (Smart Distributed and Intelligent Applications)
Parcours : SMA et IAD

PROJET DE FIN DE MODULE

Ecosystème Multi-Agent : Architecte Matériel PC

Réalisé par :

Marouane MAHBOUB

Abderahim FARAHI

Encadré par :

Pr. Sara RETAL

Code Source :

https://github.com/marouane-m7b/ENSET_PFM_Agent_Building_PC

15 mai 2026

Table des matières

1	Introduction et Choix du Sujet	2
2	Détails d'Implémentation (Technologies Utilisées)	2
3	Architecture du Système Multi-Agent	2
3.1	Architecture Modulaire	3
3.2	Agents Spécialisés	3
3.2.1	Agent Architecte (ArchitectAgent)	3
3.2.2	Agent Procurement (ProcurementAgent)	3
3.2.3	Système Human-in-the-Loop (HumanLoopWorkflow)	3
3.3	Orchestration LangGraph	3
4	Défis Techniques et Résolution des Hallucinations	4
4.1	Hallucinations de Satisfaction d'Objectif	4
4.2	Limitations de la Recherche Sémantique	4
5	Evaluation et Optimisation des Prompts	5
5.1	Framework d'Evaluation Complet	5
5.1.1	Tests A/B Automatisés	5
5.1.2	Métriques de Performance	5
5.2	Résultats de l'Optimisation	5
5.2.1	Classement des Performances	5
5.2.2	Gains de Performance	6
6	Interface Utilisateur et Human-in-the-Loop	6
6.1	Interface Wizard Interactive	6
6.2	Intégration Discord et Gestion d'Inventaire	6
6.3	Architecture Modulaire du Human-in-the-Loop	6
6.3.1	Gestionnaire d'Approbation (ApprovalManager)	7
6.3.2	Interface Utilisateur Modulaire (HumanApprovalUI)	7
6.3.3	Intégration Workflow (HumanLoopWorkflow)	7
6.4	Implémentation Technique	7
6.5	Fonctionnalités Avancées	7
6.5.1	Validation Automatique de Sécurité	7
6.5.2	Persistance et Audit	8
7	Structure du Projet	8
8	Conclusion et Perspectives	8
8.1	Réalisations Accomplies	8
8.2	Innovations Techniques	9
8.3	Impact Académique	9
8.4	Perspectives d'Amélioration	9
8.5	Validation Expérimentale	9

1 Introduction et Choix du Sujet

Pour ce projet de fin de module, nous avons choisi de développer un **Architecte Matériel PC (Custom PC Builder)**. Le choix de ce sujet n'est pas fortuit : l'assemblage d'un ordinateur est un problème complexe d'ingénierie et de satisfaction de contraintes.

Pourquoi ce sujet ?

- **Pertinence pour le Multi-Agent** : Contrairement à un chatbot classique, construire un PC nécessite plusieurs expertises distinctes : planifier un budget, vérifier la compatibilité physique (Sockets, DDR4 vs DDR5), et acheter au meilleur prix.
- **Nécessité du RAG** : Le marché du hardware évolue chaque jour. Un grand modèle de langage (LLM) seul proposerait des composants obsolètes ou hors stock. L'utilisation du RAG était indispensable pour connecter l'agent à un inventaire local en temps réel.
- **Besoin du Human-in-the-Loop** : L'achat de matériel coûtant parfois des dizaines de milliers de dirhams, il est impensable de laisser l'IA finaliser une transaction sans l'approbation humaine finale.

2 Détails d'Implémentation (Technologies Utilisées)

Le code source de notre projet a été structuré de manière modulaire autour des technologies de pointe de l'écosystème IA :

- **LangGraph** : Utilisé pour concevoir l'orchestrateur (StateGraph). Il maintient l'état global (BuilderState) et permet d'interrompre l'exécution via `interrupt_before` pour le Human-in-the-Loop.
- **Groq API (LLaMA 3.3 70B)** : Modèle de langage utilisé pour l'analyse sémantique des requêtes (Agent Architecte) et la stratégie de sélection (Agent Procurement). Choisi pour sa rapidité d'inférence.
- **ChromaDB + nomic-embed-text** : Base de données vectorielle (RAG) alimentée par les embeddings locaux d'Ollama. Nos 8 fichiers CSV d'inventaire (CPUs, GPUs, RAM, etc.) sont vectorisés et indexés par catégorie.
- **MySQL** : Base relationnelle pour la persistance de l'historique des conversations, des sessions et des décisions d'approbation.
- **Flask / Socket.IO** : Backend web temps réel. Socket.IO émet des événements (`agent_status`, `approval_needed`, `discord_sent`) pour synchroniser l'état du workflow avec le frontend.
- **Discord Webhook** : Intégration pour envoyer automatiquement les builds approuvés vers un canal Discord d'équipe.
- **Pandas** : Mécanisme de fallback déterministe si l'API Groq est indisponible (Rate Limits).

3 Architecture du Système Multi-Agent

Dans le cadre de ce projet, nous avons développé un écosystème intelligent capable de concevoir un ordinateur optimisé selon un budget précis. Le flux de travail a été décomposé hiérarchiquement avec une architecture modulaire avancée.

3.1 Architecture Modulaire

Le système a été structuré en modules indépendants pour améliorer la lisibilité, la maintenabilité et la réutilisabilité :

- **Module `agents/nodes.py`** : Contient les implémentations des agents individuels avec des classes spécialisées (`ArchitectAgent`, `ProcurementAgent`)
- **Module `human_loop/`** : Système complet de validation humaine avec gestion d'état, interface utilisateur et intégration workflow
- **Module `evaluation/`** : Framework d'évaluation et d'optimisation des prompts avec tests A/B
- **Module `rag/`** : Système de récupération augmentée avec base vectorielle ChromaDB

3.2 Agents Spécialisés

3.2.1 Agent Architecte (`ArchitectAgent`)

Responsabilités principales :

- Analyse sémantique des requêtes utilisateur
- Extraction intelligente du budget avec patterns regex multiples
- Classification automatique des cas d'usage (gaming, IA/ML, bureautique, création de contenu)
- Détermination du niveau de performance requis

3.2.2 Agent Procurement (`ProcurementAgent`)

Responsabilités principales :

- Interrogation de la base RAG avec filtrage catégoriel
- Algorithme d'optimisation avec vérification de compatibilité matérielle
- Génération de devis formatés avec métriques de performance
- Validation automatique des contraintes budgétaires

3.2.3 Système Human-in-the-Loop (`HumanLoopWorkflow`)

Responsabilités principales :

- Gestionnaire d'approbation avec états persistants
- Interface utilisateur modulaire avec composants Flask et SocketIO
- Intégration transparente avec LangGraph
- Historique et statistiques des décisions humaines

3.3 Orchestration LangGraph

L'orchestrateur utilise un `StateGraph` avec état typé (`BuilderState`) étendu pour supporter la gestion des métadonnées, le suivi des analyses et la persistance des composants.

Listing 1 – Définition du workflow LangGraph (`graph.py`)

```
class BuilderState(TypedDict):
    user_request: str
    architect_plan: str
    final_build_quote: str
    selected_components: dict
```

```

total_price: int
human_approval: str
conversation_history: List[dict]

# Construction du graphe séquentiel
workflow = StateGraph(BuilderState)
workflow.add_node("architect", architect_node)
workflow.add_node("procurement", procurement_node)
workflow.add_node("human_approval", human_approval_node)

workflow.add_edge(START, "architect")
workflow.add_edge("architect", "procurement")
workflow.add_edge("procurement", "human_approval")
workflow.add_edge("human_approval", END)

# Compilation avec checkpoint et interruption
memory = MemorySaver()
agent_app = workflow.compile(
    checkpoint=memory,
    interrupt_before=["human_approval"]
)

```

La méthode de collaboration choisie est **séquentielle** : chaque agent dépend de la sortie du précédent. Ce choix se justifie car la sélection de composants nécessite d'abord une analyse des besoins (Architecte), puis une recherche optimisée (Procurement), et enfin une validation humaine avant toute action irréversible (mise à jour du stock et notification Discord).

4 Défis Techniques et Résolution des Hallucinations

Le développement de ce système nous a confrontés à plusieurs défis d'ingénierie inhérents aux LLMs.

4.1 Hallucinations de Satisfaction d'Objectif

Le Problème : Initialement, l'Agent Procurement mentait sur les prix ou inventait des noms de composants inexistants (ex : "NVIDIA Radeon") dans le but mathématique de satisfaire parfaitement le budget imposé par l'utilisateur.

La Solution : Mise en place d'un mécanisme de Chain-of-Thought (CoT) via une balise <thinking>. En forçant l'agent à extraire les vrais prix du contexte RAG et à écrire ses calculs d'addition étape par étape avant de générer le devis final, nous avons drastiquement réduit les erreurs logiques et budgétaires.

4.2 Limitations de la Recherche Sémantique

Le Problème : La base vectorielle ChromaDB filtrait parfois des composants critiques. La similitude sémantique échouait à comprendre qu'une requête pour du "High-End" correspondait concrètement à un "Core i7" ou un "Ryzen 9".

La Solution : Passage à une Recherche Catégorielle Structurée. Au lieu d'une seule recherche globale, le système itère sur un tableau de 8 catégories (CPUs, GPUs, RAM, etc.) et effectue une récupération spécifique pour chacune.

Listing 2 – Recherche catégorielle RAG

```
categories = ["CPUS", "GPUS", "MOTHERBOARDS", "RAM",  
             "STORAGE", "COOLERS", "PSUS", "CASES"]  
full_context = []  
  
for cat in categories:  
    results = db.as_retriever(  
        search_kwargs={"k": 2, "filter": {"category": cat}}  
    ).invoke(plan)  
    for doc in results:  
        full_context.append(f"[CATEGORY: {cat}] {doc.page_content}")
```

5 Evaluation et Optimisation des Prompts

5.1 Framework d'Évaluation Complet

Nous avons développé un système d'évaluation sophistiqué pour optimiser les performances des prompts utilisés par nos agents.

5.1.1 Tests A/B Automatisés

Le framework implémente une méthodologie de tests A/B rigoureuse :

- **4 Stratégies de Prompts** : Basic, Detailed, Structured, Context-Aware
- **6 Scénarios de Test** : Gaming, Workstation, Office, AI/ML, Streaming, Budget constraint
- **Métriques Multiples** : Précision, adhérence budgétaire, compatibilité, temps de réponse, satisfaction utilisateur
- **Analyse Statistique** : Moyenne, médiane, écart-type, intervalles de confiance

5.1.2 Métriques de Performance

Le système évalue cinq dimensions critiques :

- **Précision d'Extraction** : Capacité à extraire correctement le budget et les exigences
- **Adhérence Budgétaire** : Respect des contraintes financières (tolérance plus ou moins 10 pourcent)
- **Compatibilité Matérielle** : Vérification des compatibilités CPU-Motherboard, RAM-CPU
- **Temps de Réponse** : Performance temporelle des différentes stratégies
- **Satisfaction Utilisateur** : Score composite basé sur les autres métriques

5.2 Résultats de l'Optimisation

Les tests A/B ont révélé des différences significatives entre les stratégies :

5.2.1 Classement des Performances

1. **Strategy D (Context-Aware)** : Score 0.765 - Meilleure performance globale
2. **Strategy C (Structured)** : Score 0.764 - Excellente précision (0.800)

3. **Strategy B (Detailed)** : Score 0.674 - Bon compromis performance/temps
4. **Strategy A (Basic)** : Score 0.542 - Baseline de référence

5.2.2 Gains de Performance

- **Amélioration maximale** : 41 pourcent entre la meilleure et la pire stratégie
- **Temps de réponse optimal** : Strategy A (1.75s) vs Strategy C (1.80s)
- **Précision maximale** : Strategy C avec 80 pourcent de précision d'extraction
- **Robustesse** : Strategy D excellente sur tous les scénarios de complexité

6 Interface Utilisateur et Human-in-the-Loop

6.1 Interface Wizard Interactive

L'interface utilisateur a été conçue comme un **wizard interactif** (et non un simple chatbot) composé de cinq écrans :

1. **Écran d'Accueil** : Sélection du cas d'usage (Gaming, IA/ML, Bureautique, Développement, Création de Contenu, Général) et configuration du budget via un slider interactif ou un champ de saisie directe (5 000 – 80 000 MAD).
2. **Écran de Construction IA** : Visualisation en temps réel du pipeline multi-agent (Architecte → Procurement → Approbation) avec barres de progression animées.
3. **Écran Récapitulatif** : Affichage des 8 composants sélectionnés par l'IA avec coût total, pourcentage du budget utilisé, économies et statut de compatibilité. Deux actions : *Approuver et Envoyer à Discord* ou *Modifier le Build*.
4. **Écran de Modification** : Permet de cliquer sur n'importe quel composant pour afficher **3 alternatives au prix le plus proche**, tout en respectant les contraintes de compatibilité (Socket CPU ↔ Motherboard, DDR4/DDR5 ↔ RAM).
5. **Écran de Résultat** : Confirmation de l'approbation avec mise à jour de l'inventaire et notification Discord.

6.2 Intégration Discord et Gestion d'Inventaire

Lorsque l'utilisateur approuve un build :

- Le stock de chaque composant sélectionné est décrémenté automatiquement dans les fichiers CSV via le module `inventory_manager.py`.
- Un message formaté contenant le détail complet du build est envoyé au canal Discord de l'équipe via un Webhook.
- L'historique de la décision est persisté dans la base MySQL pour audit.

6.3 Architecture Modulaire du Human-in-the-Loop

Le système de validation humaine a été entièrement refactorisé en une architecture modulaire comprenant trois composants principaux.

6.3.1 Gestionnaire d'Approbation (ApprovalManager)

Classe centrale gérant le cycle de vie des demandes d'approbation :

- **Création de requêtes** : Génération d'identifiants uniques et horodatage
- **Validation de sécurité** : Vérification automatique des configurations
- **Traitement des décisions** : Gestion des approbations/rejets avec feedback
- **Historique persistant** : Sauvegarde et export des décisions pour audit

6.3.2 Interface Utilisateur Modulaire (HumanApprovalUI)

Composants Flask/SocketIO spécialisés pour l'interaction humaine :

- Interface d'approbation avec boutons contextuels
- Comparaison requête/build côte-à-côte
- Collecte de feedback avec zone de texte
- Statistiques en temps réel et historique

6.3.3 Intégration Workflow (HumanLoopWorkflow)

Pont entre le système d'approbation et LangGraph :

- Génération automatique de nœuds LangGraph
- Synchronisation entre l'état workflow et les requêtes
- Nettoyage automatique des requêtes expirées
- Sauvegarde et restauration de l'état workflow

6.4 Implémentation Technique

Listing 3 – Intégration modulaire du Human-in-the-Loop

```
# Initialisation du systeme modulaire
from human_loop import (
    approval_manager,
    create_approval_ui,
    initialize_workflow_integration
)

# Configuration de l'integration workflow
workflow_integration = initialize_workflow_integration(
    approval_manager
)
approval_ui = create_approval_ui(approval_manager)

# Creation du noeud d'approbation pour LangGraph
human_approval_node = workflow_integration.create_human_approval_node()

# Integration dans le workflow
workflow.add_node("human_approval", human_approval_node)
```

6.5 Fonctionnalités Avancées

6.5.1 Validation Automatique de Sécurité

Le système effectue des vérifications automatiques :

- Validation budgétaire avec tolérance de 10 pourcent
- Contrôle de cohérence des composants essentiels
- Vérification de format du devis généré
- Détection d'anomalies dans les prix

6.5.2 Persistance et Audit

- Export JSON avec sauvegarde automatique des décisions
- Statistiques d'approbation (taux, temps de décision moyen)
- Traçabilité complète avec historique et feedback
- Nettoyage automatique des requêtes expirées

7 Structure du Projet

Le projet est organisé de manière modulaire :

Dossier agents/ : Système multi-agent

- graph.py : Orchestration LangGraph
- nodes.py : Implémentations des agents

Dossier human_loop/ : Système Human-in-the-Loop

- approval_manager.py : Gestionnaire d'approbation
- ui_components.py : Interface utilisateur
- workflow_integration.py : Intégration workflow

Dossier evaluation/ : Framework d'évaluation

- prompt_evaluation.py : Tests de performance
- ab_testing.py : Tests A/B
- metrics_dashboard.py : Dashboard de monitoring

Dossier rag/ : Système RAG

- ingest.py : Ingestion des données
- chroma_db/ : Base vectorielle

Dossier data/ : Bases de données composants (8 fichiers CSV : cpus, gpus, motherboards, ram, storage, coolers, psus, cases)

Dossier database/ : Gestionnaire MySQL (db_manager.py) pour la persistance

Fichier app.py : Serveur Flask/Socket.IO avec APIs REST (/api/components/recommend, /api/components/alternatives) et handlers Socket.IO

Fichier inventory_manager.py : Gestion du stock (décrémentation automatique après approbation)

8 Conclusion et Perspectives

8.1 Réalisations Accomplies

Ce projet démontre la puissance d'une architecture multi-agent modulaire et orchestrée. Nous avons réussi à implémenter tous les composants requis :

- **Orchestration Multi-Agent** : Architecture LangGraph avec agents spécialisés et modulaires
- **RAG Agentique** : Système de récupération augmentée avec base vectorielle ChromaDB

- **Human-in-the-Loop Avancé** : Système complet de validation avec interface modulaire
- **Interface Web Interactive** : Application Flask/Socket.IO avec wizard interactif, visualisation du pipeline multi-agent en temps réel, et mécanisme de modification avec alternatives
- **Evaluation des Prompts** : Framework complet avec tests A/B et optimisation

8.2 Innovations Techniques

- **Architecture Modulaire** : Séparation claire des responsabilités en modules indépendants
- **Système d'Approbation Sophistiqué** : Gestion d'état persistant et validation automatique
- **Interface Wizard** : Expérience utilisateur guidée avec alternatives intelligentes (3 composants au prix le plus proche avec filtrage de compatibilité)
- **Optimisation Basée sur les Données** : Tests A/B avec amélioration de 41 pourcent
- **Intégration Discord** : Notification automatique de l'équipe avec Webhook
- **Robustesse Opérationnelle** : Mécanismes de fallback Pandas et gestion d'erreurs

8.3 Impact Académique

Le système illustre l'application pratique des concepts théoriques du cours SMA et IAD :

- Coordination Multi-Agent avec orchestration séquentielle
- Interaction Homme-Machine avec validation humaine
- Optimisation Algorithmique par évaluation empirique
- Ingénierie Logicielle avec architecture maintenable

8.4 Perspectives d'Amélioration

- Agent Vérificateur Indépendant pour simulation de benchmarks
- Optimisation Multi-Objectifs avec algorithmes génétiques
- Apprentissage Adaptatif basé sur l'historique des approbations
- Intégration E-commerce avec APIs de fournisseurs
- Système de Recommandation avec ML pour suggestions personnalisées

8.5 Validation Expérimentale

Les tests A/B ont démontré l'efficacité de notre approche avec des améliorations mesurables de 41 pourcent entre les stratégies de prompts, validant scientifiquement notre méthodologie d'optimisation.

Lien vers le code source (GitHub) :

https://github.com/marouane-m7b/ENSET_PFM_Agent_Building_PC